ParalESN: Enabling parallel information processing in Reservoir Computing

Matteo Pinna [*1] **Giacomo Lagomarsini** [*1] **Andrea Ceni** [1] **Claudio Gallicchio** [1]

Abstract

Reservoir Computing (RC) has established itself as an efficient paradigm for temporal processing. However, its scalability remains severely constrained by (i) the necessity of processing temporal data sequentially and (ii) the prohibitive memory footprint of high-dimensional reservoirs. In this work, we revisit RC through the lens of structured operators and state space modeling to address these limitations, introducing Parallel Echo State Network (ParalESN). ParalESN enables the construction of high-dimensional and efficient reservoirs based on diagonal linear recurrence in the complex space, enabling parallel processing of temporal data. We provide a theoretical analysis demonstrating that ParalESN preserves the Echo State Property and the universality guarantees of traditional Echo State Networks while admitting an equivalent representation of arbitrary linear reservoirs in the complex diagonal form. Empirically, ParalESN matches the predictive accuracy of traditional RC on time series benchmarks, while delivering substantial computational savings. On 1-D pixel-level classification tasks, ParalESN achieves competitive accuracy with fully trainable neural networks while reducing computational costs and energy consumption by orders of magnitude. Overall, ParalESN offers a promising, scalable, and principled pathway for integrating RC within the deep learning landscape.

1. Introduction

Reservoir Computing (RC) has emerged as a simple yet powerful paradigm for harnessing the rich dynamics of recurrent systems for learning and prediction (Nakajima & Fischer, 2021; Lukoševičius & Jaeger, 2009). By fixing a non-linear recurrent reservoir and training only a linear readout, RC offers favorable training efficiency, strong performance on temporal processing tasks, and intriguing connections to both neuroscience and dynamical systems theory. These properties have led to its success in a wide range of domains, from speech recognition to chaotic signal prediction. Despite these strengths, RC faces the same problem of traditional, fully-trainable Recurrent Neural Networks (RNNs): the input signal has to be processed sequentially, which makes training slow and not parallelizable. In an attempt to improve the efficiency of RC, structured operators have been investigated (Rodan & Tino, 2011; Dong et al., 2020; D'Inverno & Dong, 2025). A particularly active research area involves exploring RC implementations in hardware implementations (Gallicchio & Soriano, 2025). An important theoretical insight is that even *linear* reservoirs, provided that the readout is expressive enough, are universal approximators in the class of fading memory filters (Grigoryeva & Ortega, 2018a;b), thus being able to represent well arbitrary input-output dynamics.

In this work, we address limitations of classical RC by introducing Parallel Echo State Networks (ParalESN), a novel class of efficient untrained RNNs with diagonal linear recurrence in the complex space, where the recurrence can be parallelized via associative scan. Fig. 1 graphically highlights the proposed model. Our approach rethinks reservoir construction through the lens of structured operators. By combining the dynamical richness of RC with the linear recurrence from Linear Recurrent Unit (LRU) (Orvieto et al., 2023), we bridge a gap between classical dynamical systems-inspired learning and contemporary large-scale modeling.

The remainder of this work is organized as follows. Section 2 discusses related works. Section 3 presents the proposed approach, ParalESN. Section 4 presents our theoretical analysis. Section 5 presents the experiments. Finally, Section 6 concludes the paper. In the Appendix, we provide mathematical proofs, definitions, and additional experiments. See Appendix A for a table of contents.

2. Related works

Reservoir Computing. Reservoir Computing (RC) (Verstraeten et al., 2007; Nakajima & Fischer, 2021) is a pop-

[*]Equal contribution. [1]Department of Computer Science, University of Pisa, Italy. Correspondence to: Matteo Pinna <matteo.pinna@di.unipi.it>, Giacomo Lagomarsini <giacomo.lagomarsini@phd.unipi.it>.

Preprint. February 2, 2026.

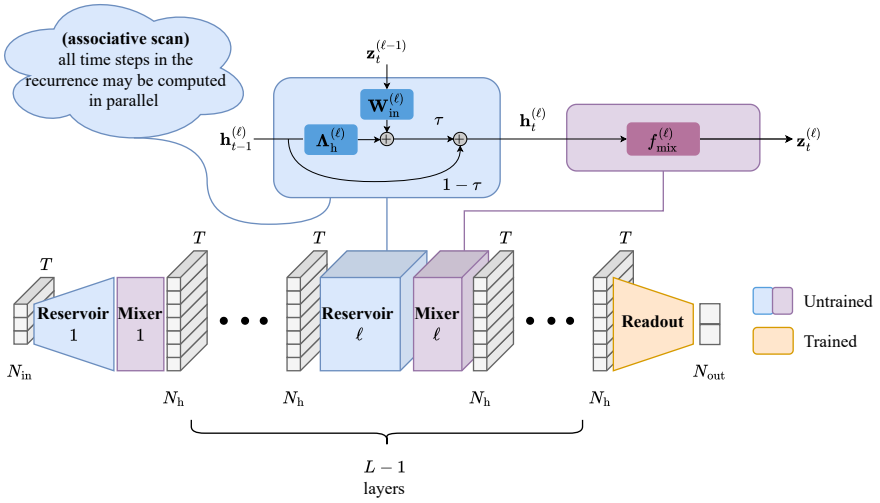

Figure 1. Architectural organization of the proposed ParalESN. The model may have multiple blocks consisting of two components: (i) a linear reservoir and (ii) a non-linear mixing layer. The first block processes the external input as in a traditional shallow architecture. Subsequent blocks process the output of the previous block's mixing layer, $\mathbf{z}_t^{(\ell-1)}$. The structure of the reservoir (in blue) includes a diagonal, complex-valued transition matrix $\mathbf{\Lambda}_{\text{h}}^{(\ell)}$ and a dense complex-valued input weight matrix $\mathbf{W}_{\text{in}}^{(\ell)}$. The main branch and the temporal residual connections are scaled by a positive coefficient τ and $1-\tau$, respectively. The mixing layer (in purple) is used to introduce non-linearity in the model's dynamics and to mix the components of the reservoir states at each time step. The readout (in orange) is the only trainable component in the system. See Section 3 for details.

ular framework for the design of efficient untrained Recurrent Neural Networks (RNNs), developed to address the instabilities of training RNNs (Bengio et al., 1994; Glorot & Bengio, 2010; Pascanu et al., 2013). An RC model consists of two main components: (i) a high-dimensional recurrent layer, called *reservoir*, that is randomly initialized and then left untrained, and (ii) a trainable readout layer, that may be trained via lightweight closed-form solutions. Therefore, by design, RC models bypass backpropagation and related vanishing/exploding gradients, and can be trained exceptionally fast in a single forward pass.

Echo State Networks (ESNs) (Jaeger et al., 2007) established themselves as one of the most successful instance of RC models. ESNs dynamics can be defined as follows:

$$\mathbf{h}_t = (1-\tau)\mathbf{h}_{t-1} + \tau\,\sigma(\mathbf{W}_{\text{h}}\mathbf{h}_{t-1} + \mathbf{W}_{\text{in}}\mathbf{x}_t + \mathbf{b}). \quad (1)$$

where $\mathbf{h}_t \in \mathbb{R}^{N_{\text{h}}}$ and $\mathbf{x}_t \in \mathbb{R}^{N_{\text{in}}}$ are, respectively, the state and the external input at time step t. The transition matrix is denoted as $\mathbf{W}_{\text{h}} \in \mathbb{R}^{N_{\text{h}} \times N_{\text{h}}}$, the input weight matrix is denoted as $\mathbf{W}_{\text{in}} \in \mathbb{R}^{N_{\text{h}} \times N_{\text{in}}}$, $\mathbf{b} \in \mathbb{R}^{N_{\text{h}}}$ denotes the bias vector, σ denotes an element-wise applied non-linearity, and $\tau \in (0, 1]$ denotes the leaky rate hyperparameter. The entries of the matrix $\mathbf{W}_{\text{in}}$ and $\mathbf{b}$ are generally sampled randomly from a uniform distribution (Gallicchio et al., 2017; Ceni & Gallicchio, 2024) over $[-\omega_{\text{in}}, \omega_{\text{in}}]$ and $[-\omega_{\text{b}}, \omega_{\text{b}}]$, respectively. Sampling their entries from Gaussian distributions is also popular (Verstraeten et al., 2007). The entries of $\mathbf{W}_{\text{h}}$ are randomly sampled from a uniform distribution over $[-1, 1]$ and then rescaled to have a desired spectral radius ρ[1]. In practical applications, the spectral radius is generally constrained to be smaller than 1.

The final output is retrieved through the linear readout:

$$\mathbf{y}_t = \mathbf{W}_{\text{out}}\mathbf{h}_t + \mathbf{b}_{\text{out}}. \quad (2)$$

where $\mathbf{y}_t \in \mathbb{R}^{N_{\text{out}}}$ denotes the network output at time step t, and $\mathbf{W}_{\text{out}} \in \mathbb{R}^{N_{\text{out}} \times N_{\text{h}}}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^{N_{\text{out}}}$ denote the readout weight matrix and bias vector, respectively. The readout is typically optimized via lightweight closed-form solutions, e.g. ridge regression or least squares methods.

The model's state update function defined in (1) could depend, in principle, from the initial point $\mathbf{h}_0$. This leads to non-deterministic behaviors, i.e. the impossibility to determine the state solely from the inputs. To avoid that, the reservoir in ESNs is initialized subject to the Echo State Property (ESP) (Yildiz et al., 2012), a useful stability condition for guiding ESN initialization. We say that a discrete time dynamical system defined by the transition function $F(\mathbf{h}, \mathbf{x})$ satisfies the ESP if the states asymptotically depends only on the system's inputs. In other words, for any

[1]The spectral radius of a matrix $\mathbf{A}$, denoted $\rho(\mathbf{A})$, is defined as the largest among the lengths of its eigenvalues.

two initial points $\mathbf{h}_0$ and $\mathbf{h}'_0$, then

$$\lim_{t\to\infty} \|F(\mathbf{h}_{t-1}, \mathbf{x}_t) - F(\mathbf{h}'_{t-1}, \mathbf{x}_t)\|_2 = 0. \quad (3)$$

Several ESN variants lay their foundation at the intersection of the RC and DL frameworks. In particular, Deep Echo State Networks (DeepESNs) (Gallicchio et al., 2017) generalize the concept of shallow ESNs towards deep architectural constructions, where multiple untrained reservoirs are stacked on top of each other. The increased feedforward depth has been shown to provide advantageous architectural bias relative to shallow ESNs. More recently, Residual Echo State Networks (ResESNs) (Ceni & Gallicchio, 2024) introduced orthogonal residual connections along the temporal dimension to enhance long-term information processing capabilities. Other works have explored structured transforms to improve the efficiency of RC, including Simple Cycle Reservoir (SCR) (Rodan & Tino, 2011), where the transition matrix employs a fixed ring topology, and Structured Reservoir Computing (Structured RC) (Dong et al., 2020), where the transition matrix consists of a composition of Hadamard and diagonal matrices.

Universality of linear reservoirs. ESP guarantees the universality of the reservoir system in approximating fading memory filters (Grigoryeva & Ortega, 2018a). Provided that the ESP holds, similar universality results have also been proven for reservoirs characterized by linear dynamics, when combined with non-linear readouts that universally approximate functions $f : \mathbb{C}^{N_\mathrm{h}} \to \mathbb{C}^{N_y}$ (Grigoryeva & Ortega, 2018b; Gonon & Ortega, 2020). Therefore, linear recurrence ESNs, in combination with non-linear readouts, can approximately arbitrarily well any time-invariant causal filter with the fading memory property.

Transformers. Transformers are the de-facto standard architecture for sequence modeling, managing to replace recurrent models on real-world tasks since their introduction in (Vaswani et al., 2017). Differently from RNNs, transformers are a feedforward architecture, and employ self-attention, enabling access to every position of the sequence at any time. This allows for great parallelization and modeling capacity, but comes at a cost of a quadratic complexity in sequence length, making scaling to long contexts challenging. To address this issue, many variants to the naive self-attention aiming to lower the computational and memory burden while maintaining expressivity have been proposed (Tay et al., 2022).

State Space Models. Besides training instabilities, another major drawback of stateful sequence models like RNNs is that the input needs to be processed sequentially, greatly limiting the parallelization capabilities of these type of architectures on modern accelerators. One of the most promising approaches for the parallelization of recurrent models is that of (Deep) State Space Models (SSMs) (Gu et al., 2021). SSMs start from the idea of a linear state space dynamical system, and devise initialization and discretization strategies that help improve memory capacity of the system. Linear recurrence is easily parallelizable (see e.g. Martin & Cundy, 2018), greatly improving training and inference efficiency of this family of models. Structured SSMs such as S4 and S5 (Gu et al., 2021; Smith et al., 2022) demonstrated how linear recurrence, along with a careful initialization based on HiPPO matrices (Gu et al., 2020), can enhance long memory propagation on long sequence tasks. Building on these foundations, Mamba (Gu & Dao, 2024; Dao & Gu, 2024) proposes a selective architecture that allows to change the dynamic based on its inputs. These advances position SSMs as contenders to transformers in sequence modeling, particularly in tasks demanding long memory retention.

Linear recurrent unit. Linear recurrent Unit (LRU) (Orvieto et al., 2023) successfully applied linear recurrence to general RNNs, demonstrating that it is possible to deviate from the strict initialization and parametrization rules of SSMs, while retaining their impressive performance. Rather than initializing matrices using HiPPO theory as standard SSMs, LRU employs a diagonal transition matrix, whose eigenvalues are initialized inside the unitary complex disk, using a de-coupled parametrization of their magnitude and phase. In particular, the magnitude is chosen in an interval $[r_{\min}, r_{\max}]$, which allows a finer control over stability and memory capacity of the model. The linear, diagonal structure reduces recurrent updates to parallelizable element-wise operations, and the initialization strategy makes the architecture more stable for longer sequences.

3. ParalESN

Inspired by the success of SSMs, we design a more scalable and efficient reservoir architecture, which we call *ParalESN*. ParalESN improves upon previous RC models by employing a linear diagonal recurrence, similar to that of LRU, followed by a mixing function that combines reservoir states. As is standard in the RC paradigm, only the readout function is trainable. We also explore a *deep* variant of ParalESN, which we denote ParalESN (deep). See Fig. 1 for a graphical representation of the architecture.

Let $\mathbf{z}_t^{(0)} = \mathbf{x}_t$ denote the input to the first layer. The reservoir dynamics at layer ℓ are described by:

$$\begin{aligned}\mathbf{h}_t^{(\ell)} =& (1 - \tau^{(\ell)})\, \mathbf{h}_{t-1}^{(\ell)} \\ &+ \tau^{(\ell)} \left(\mathbf{\Lambda}_\mathrm{h}^{(\ell)} \mathbf{h}_{t-1}^{(\ell)} + \mathbf{W}_\mathrm{in}^{(\ell)} \mathbf{z}_t^{(\ell-1)} + \mathbf{b}^{(\ell)} \right), \end{aligned} \quad (4)$$

where superscript (ℓ) denotes layer-specific hyperparameters and weights. For each time step $t \in \{1, \dots, T\}$, $\mathbf{x}_t \in \mathbb{R}^{N_\mathrm{in}}$ is the external input, $\mathbf{h}_t^{(\ell)} \in \mathbb{C}^{N_\mathrm{h}}$ is the reservoir state, and $\mathbf{z}_t^{(\ell)} \in \mathbb{R}^{N_\mathrm{h}}$ is the hidden state after the mixing

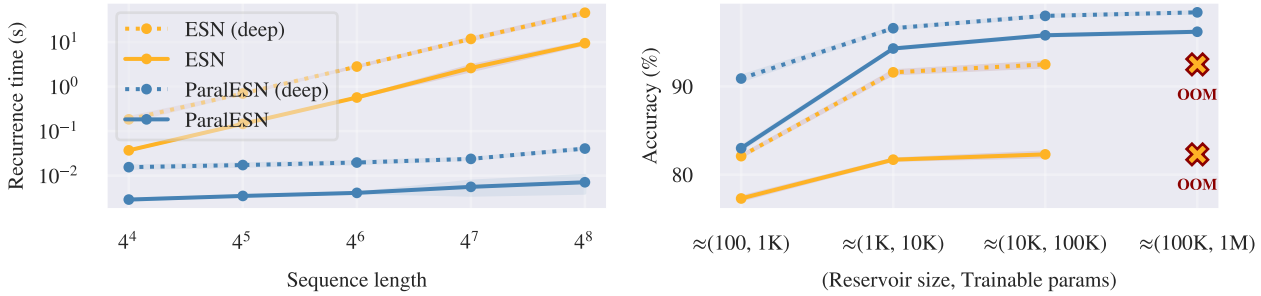

Figure 2. (*left*) Time required to perform the recurrence in ParalESN and traditional ESNs for increasing sequence lengths, assuming 128 recurrent neurons and 5 layers for deep configurations. ParalESN scales logarithmically with sequence length, whereas traditional ESNs scale linearly. (*right*) Scaling ParalESN and traditional ESNs to high-dimensional reservoirs on the sMNIST task. Traditional ESNs run out-of-memory (OOM) at approximately 100K reservoir neurons, while ParalESN fit into memory due to the reduced memory footprint of their diagonal transition matrices.

step. The matrix $\mathbf{\Lambda}_{\text{h}}^{(\ell)} \in \mathbb{C}^{N_{\text{h}} \times N_{\text{h}}}$ is a diagonal transition matrix, $\mathbf{b}^{(\ell)} \in \mathbb{C}^{N_{\text{h}}}$ is the bias vector, and $\tau^{(\ell)} \in (0, 1]$ is the leaky rate. Since the recurrence is linear, the leaky integration can be partially absorbed into the transition matrix. Accounting for leakage, the effective transition matrix becomes $\bar{\mathbf{\Lambda}}^{(\ell)}_{\text{h}} = (1 - \tau^{(\ell)})\mathbf{I} + \tau^{(\ell)}\mathbf{\Lambda}^{(\ell)}_{\text{h}}$, where $\mathbf{I}$ is the identity matrix. The input weight matrix $\mathbf{W}_{\text{in}}^{(1)} \in \mathbb{C}^{N_{\text{h}} \times N_{\text{in}}}$ is dense for the first layer, mapping the external input to the hidden dimension. For subsequent layers, the input weight matrix $\mathbf{W}_{\text{in}}^{(\ell>1)} \in \mathbb{C}^{N_{\text{h}} \times N_{\text{h}}}$ employs the following ring topology (Rodan & Tino, 2011; Verzelli et al., 2020; Tino, 2020; Pinna et al., 2025) to reduce memory overhead:

$$\mathbf{W}_{\text{in}}^{(\ell>1)} = \begin{bmatrix} 0 & 0 & \cdots & 0 & w_1 \\ w_2 & 0 & \cdots & 0 & 0 \\ 0 & w_3 & \cdots & 0 & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \cdots & w_{N_{\text{h}}} & 0 \end{bmatrix}, \tag{5}$$

The matrix in (5) shifts the input vector and then applies an element-wise scaling. Consequently, we only store a N_{h}-dimensional vector of scaling coefficients, significantly reducing the memory footprint in deeper layers. The entries of the input weight matrices are initialized by sampling the real and imaginary parts independently from a uniform distribution over $[-1, 1]$ and then scaling each row i of the matrix by $\sqrt{1 - |\lambda_i|^2}$, where λ_i is the corresponding diagonal element (eigenvalue) of the transition matrix $\mathbf{\Lambda}^{(\ell)}_{\text{h}}$. The bias vectors are sampled from a uniform distribution and scaled by hyperparameter $\omega_{\text{b}}^{(\ell)}$. The diagonal elements of the transition matrix are initialized to control the eigenvalue distribution, with spectral radii sampled uniformly from $[\rho_{\text{min}}^{(\ell)}, \rho_{\text{max}}^{(\ell)}]$ and phases from $[\theta_{\text{min}}^{(\ell)}, \theta_{\text{max}}^{(\ell)}]$, forming complex eigenvalues $\rho^{(\ell)} e^{i\theta^{(\ell)}}$.

The mixing function f_{mix} introduces non-linearity into the model and enables interaction among the components of the hidden state, which would otherwise evolve independently due to the diagonal structure of the recurrence. The mixing function is defined as:

$$\mathbf{z}_t^{(\ell)} = f_{\text{mix}}^{(\ell)}(\mathbf{h}_t^{(\ell)}) = \tanh\left(\Re\left(\mathbf{W}_{\text{mix}}^{(\ell)} * \mathbf{h}_t^{(\ell)} + \mathbf{b}_{\text{mix}}^{(\ell)}\right)\right), \tag{6}$$

where $*$ denotes the convolution operator, $\mathbf{W}_{\text{mix}}^{(\ell)} \in \mathbb{C}^k$ is a 1-D kernel of size $k^{(\ell)}$, $\mathbf{b}_{\text{mix}}^{(\ell)} \in \mathbb{C}$ is a scalar bias, and $\Re(\cdot)$ extracts the real part. The kernel slides across the hidden dimension of $\mathbf{h}_t^{(\ell)} \in \mathbb{C}^{N_{\text{h}}}$ with same-padding, producing an output of identical dimension. We employ a 1-D convolution rather than a dense matrix, to reduce the memory footprint of the mixing function. The same kernel is shared across all time steps, requiring only $k + 1$ parameters regardless of sequence length or hidden size. The kernel and bias entries are sampled randomly from a uniform distribution over $[\text{-}\omega_{\text{mix}}^{(\ell)}, \omega_{\text{mix}}^{(\ell)}]$ and $[\text{-}\omega_{\text{mixb}}^{(\ell)}, \omega_{\text{mixb}}^{(\ell)}]$, respectively.

The readout layer aggregates the mixed states from all L layers:

$$\mathbf{y}_t = f_{\text{readout}}(\mathbf{z}_t^{(1)}, \ldots, \mathbf{z}_t^{(L)}) \tag{7}$$

For classification tasks, only the final time step is used, $\mathbf{y} = f_{\text{readout}}(\mathbf{z}_T^{(1)}, \ldots, \mathbf{z}_T^{(L)})$.

Fig. 2 graphically demonstrates ParalESN's speed advantage over traditional ESNs with respect to sequence length (left) and its ability to scale to high-dimensional reservoirs (right). ParalESN's recurrence time scales logarithmically with sequence length, rather than linearly, due to its ability to parallelize the recurrence. Note that even the deep configuration of ParalESN is consistently faster than the shallow configuration of ESN, despite consisting of a higher number of reservoir layers. Additionally, by employing a diagonal transition matrix, ParalESN can scale to higher-dimensional reservoirs compared to traditional RC, which generally employs dense $N_{\text{h}} \times N_{\text{h}}$ transition matrices. See Appendix B for a computational complexity analysis of

ParalESN and other RC approaches. ParalESN achieves lower time complexity, reducing it from linear to logarithmic with respect to sequence length, and the memory footprint of its parameters doesn't scale quadratically with respect to the hidden size (see Table 5).

4. Theoretical Analysis

In this section, we derive a simple condition for the ESP to hold. The ESP is necessary to prove a result of universality for the class of linear reservoir models (Grigoryeva & Ortega, 2018a).

We consider a one-layer ParalESN. The sequence of hidden states produced by the model given a sequence of inputs $\{\mathbf{x}_1, \dots, \mathbf{x}_T\} \in (\mathbb{C}^{N_{\text{in}}})^T$ and a starting point $\mathbf{h}_0 \in \mathbb{C}^{N_{\text{h}}}$ is $(\mathbf{y}_0, ..., \mathbf{y}_T)$, with

$$\mathbf{h}_t = \begin{cases} \mathbf{h}_0 & \text{if } t = 0 \\ \bar{\mathbf{\Lambda}}_{\text{h}} \mathbf{h}_{t-1} + \tau(\mathbf{W}_{\text{in}} \mathbf{x}_t + \mathbf{b}) & \text{oth.} \end{cases} \tag{8}$$

$$\mathbf{y}_t = f_{\text{out}}(\mathbf{h}_t), \tag{9}$$

where f_{out} is the readout function. Because the recurrence is linear, the readout function must be non-linear to preserve expressivity. This definition apparently differs slightly from equations (4), (6), and (7), but since training does not come into play in this section, we can incorporate f_{mix} and f_{readout} in a single function f_{out}.

4.1. Echo State Property

As a first step of the theoretical analysis, we show that it is easy to derive a simple condition for the ESP to hold in the case of linear ESN of (8). The same condition on the spectral radius that is necessary for the ESP of ESNs (see, e.g., (Jaeger & Haas, 2004)) is also sufficient in our case. Moreover, in the case of a diagonal transition matrix, we can directly control the spectral radius by examining the largest diagonal value in modulus.

Theorem 4.1 (Sufficient and necessary conditions for the ESP)**.** *A ParalESN has the ESP if and only if the diagonal elements $\lambda_1, ..., \lambda_{N_h}$ of the transition matrix $\bar{\mathbf{\Lambda}}_h$ are such that for each i, $|\lambda_i| < 1$, where $|\cdot|$ is the complex modulus.*

The proof is given in Appendix C.1.

4.2. Expressivity of ParalESN

An ESN with linear recurrence and MLP readout and the ESP property is universal in the family of fading memory filters, and in particular it is as powerful as non-linear ESNs, that have the same property (Grigoryeva & Ortega, 2018a;b) (see also Appendix D for definitions on fading memory filters). We have stated the conditions for ParalESN to have the ESP property in Theorem 4.1. We now show that an ParalESN is as expressive as a ESN with linear recurrence and any arbitrary transition matrix $\mathbf{W}_{\text{h}} \in \mathbb{C}^{N_{\text{h}} \times N_{\text{h}}}$ and MLP readout.

Proposition 4.2. *Consider an ESN with linear recurrence and a 1-layer MLP readout, defined by*

$$\begin{cases} \mathbf{h}_t = \mathbf{W}_{\text{h}} \mathbf{h}_{t-1} + \mathbf{W}_{\text{in}} \mathbf{x}_t \\ \mathbf{y}_t = \mathbf{W}_{\text{out}}(\sigma(\mathbf{W}_{\text{h}} \mathbf{h}_t)) = MLP(\mathbf{h}_t) \end{cases} \tag{10}$$

where $\mathbf{W}_{\text{h}}$, $\mathbf{W}_{\text{in}}$ are matrices of appropriate sizes. Then, there exists a ParalESN with MLP readout, defined by (8) and (9) with f_{out} being another MLP, such that for any given input, the two models produce the same output.

The proposition can be proven by diagonalizing the full matrix $\mathbf{W}_{\text{h}}$. The proof is given in Appendix C.2.

By Proposition 4.2 and results on equivalent expressiveness between ESNs with linear recurrence and standard ESNs, we can deduce that ParalESN and standard ESNs (both with the ESP) are *equivalently expressive*. We explicitly state this fact in the following corollary.

Corollary 4.3. *The class of ParalESN models with the ESP, endowed with an MLP readout, is universal in the family of fading memory filters.*

In practice, in our experiments, the MLP readout is often decomposed into two layers: the first layer, together with the non-linearity, is treated as the fixed mixing function, the last layer is treated as the trainable readout.

5. Experiments

We empirically validate the performance of the proposed approach on time series regression and classification tasks (see also Appendix E), and with respect to traditional RC and fully-trainable sequence models. As our fully-trainable models, we consider a LSTM, a standard Transformer (Vaswani et al., 2017), a Linear Recurrent Unit (LRU) (Orvieto et al., 2023) [2], and Mamba (Gu & Dao, 2024) [3] to cover a wide range of model classes, including recurrent neural networks, attention-based neural networks, and deep state space models, respectively. See Appendix F for details on the experimental setting and Appendix G for experiments on hyperparameters' sensitivity and comparison with other structured RC approaches.

Comparison with respect to traditional RC. Fig. 3 compares ParalESN provides with respect to traditional RC from performance and efficiency perspectives. The left panels presents the trade-off between performance and efficiency, the right panel presents a critical difference plot ranking models based on their overall performance. We

[2] Code from github.com/NicolasZucchet/minimal-LRU.
[3] Code from github.com/state-spaces/mamba.

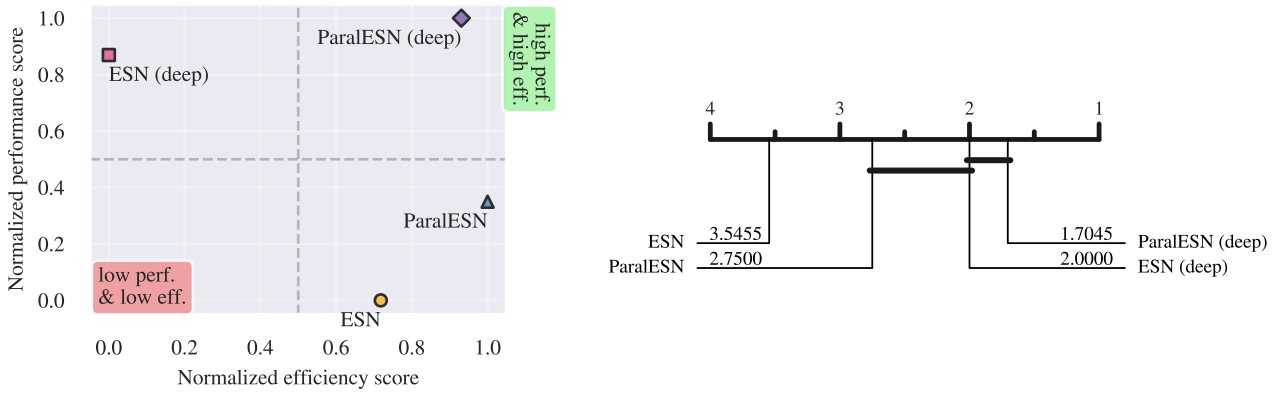

Figure 3. (*left*) Analysis of the trade-off between performance (error for regression tasks and accuracy for classification tasks) and efficiency (training time) for ParalESN and traditional ESNs across all considered benchmarks. For each model, we compute the percentage improvement over the ESN baseline for each task. The normalized scores are then obtained via min-max normalization of these average improvements, mapping them to a $[0, 1]$ scale, where 0 corresponds to the worst-performing model and 1 to the best-performing one. Overall, ParalESN and ParalESN (deep) outperform their counterparts while being more efficient. (*right*) Critical difference plot computed via a Wilcoxon test (Demšar, 2006), showing the average rank (lower is better). Models are ranked based on their overall performance across all benchmarks. Cliques (solid lines) connect models for which there is no statistically significant difference in performance. On average, ParalESN (deep) is the top-performing model.

observe that ParalESN is both more accurate and more efficient with respect to its counterpart (shallow or deep). In particular, ParalESN (deep), despite consisting of multiple reservoir layers, is able to stay competitive in terms of recurrence speed with respect to a shallow ESN. Additionally, ParalESN outperforms its shallow counterpart by a statistically significant margin. Although there is not statistically significant difference between the performance of ParalESN (deep) and ESN (deep), the former is the top-performing model while being considerably more efficient.

5.1. Time series regression

Memory-based tasks are designed to assess the model's ability to recall delayed versions of the input, while forecasting tasks evaluate the model's ability to predict future time steps. We consider MemCap (Jaeger, 2001), ctXOR (Verstraeten et al., 2010), and SinMem (Inubushi & Yoshimura, 2017) in our memory-based benchmark. For our forecasting benchmark, we consider Lorenz96 (Lorenz, 1996), Mackey-Glass (MG) (Jaeger & Haas, 2004), NARMA, and a selection of real-world time series forecasting tasks from (Zhou et al., 2021), including ETTh1, ETTh2, ETTm1, and ETTm2.

Discussion. Table 1 and Table 2 present the test set results on memory-based and forecasting tasks, respectively. Fig. 4 graphically compares the training time of ParalESN to that to traditional ESNs. Our experiments demonstrate that ParalESN can achieve comparable results to traditional RC across a wide range of time series regression benchmarks, while offering exceptional advantages from a computational efficiency perspective. In all benchmarks, ParalESN trains an entire order of magnitude faster, except for Lorenz25 and Lorenz50, where the relatively small sequence length reduces the advantage of being able to parallelize the recurrence. Observe that even ParalESN (deep), despite consisting of multiple reservoir layers, trains faster than a traditional, shallow ESN consisting of just one layer. Indeed, while the readout layer is trained via Ridge regression in both cases, the time required to perform the recurrence through the untrained reservoir is significantly lower in the proposed approach compared to traditional ESNs, thanks to being able to process the sequence in parallel.

5.2. Time series classification

The time series classification benchmark consists of a selection of tasks from the UEA & UCR repository (Bagnall et al., 2018; Dau et al., 2019). For 1-D pixel-level classification, we consider two variations of the MNIST dataset (LeCun, 1998): (i) sequential MNIST (sMNIST), where pixels are flattened into a one-dimensional vector, and (ii) permuted sequential MNIST (psMNIST), where on top of the flattening a random permutation is applied to the pixels.

Discussion. Tables 3 and 4 present test set results on time series classification tasks from the UEA & UCR repository and on sMNIST and psMNIST, respectively. Fig. 5 visualizes the trade-off between performance and efficiency among all models for the MNIST benchmarks. On time series classification, ParalESN achieves higher test accuracy compared to a shallow ESN: $+27.9\%$ on Blink, $+3.7\%$ on FaultDetectionA, $+7.6\%$ on FordA, $+5.7\%$ on FordB, and

Table 1. Test set results of memory-based tasks, assuming 128 recurrent neurons for each model. The **best result** is highlighted in bold.

MEMORY-BASED	MEMCAP (↑)	$\cdot 10^{-1}$ CTXOR5 (↓)	$\cdot 10^{-1}$ CTXOR10 (↓)	$\cdot 10^{-1}$ SINMEM10 (↓)	$\cdot 10^{-1}$ SINMEM20 (↓)
ESN	$50.6_{\pm 1.6}$	$3.6_{\pm 0.1}$	$7.7_{\pm 0.6}$	$3.6_{\pm 0.1}$	$3.7_{\pm 0.1}$
ESN (deep)	$56.8_{\pm 1.3}$	$\mathbf{3.4_{\pm 0.2}}$	$\mathbf{5.2_{\pm 1.0}}$	$1.2_{\pm 0.1}$	$\mathbf{1.6_{\pm 0.1}}$
ParalESN	$114.5_{\pm 1.4}$	$3.9_{\pm 0.1}$	$8.2_{\pm 0.2}$	$3.7_{\pm 0.0}$	$3.7_{\pm 0.0}$
ParalESN (deep)	$\mathbf{125.0_{\pm 0.2}}$	$3.6_{\pm 0.1}$	$5.6_{\pm 0.4}$	$\mathbf{1.0_{\pm 0.2}}$	$2.5_{\pm 0.4}$

Table 2. Test set results of forecasting tasks, assuming 128 recurrent neurons for each model. The **best result** is highlighted in bold.

FORECASTING	$\cdot 10^{-2}$ LZ25 (↓)	$\cdot 10^{-2}$ LZ50 (↓)	$\cdot 10^{-4}$ MG (↓)	$\cdot 10^{-2}$ MG84 (↓)	$\cdot 10^{-2}$ N10 (↓)	$\cdot 10^{-2}$ N30 (↓)	$\cdot 10^{-1}$ ETTH1 (↓)	$\cdot 10^{-1}$ ETTH2 (↓)	$\cdot 10^{-1}$ ETTM1 (↓)	$\cdot 10^{-1}$ ETTM2 (↓)
ESN	$10.0_{\pm 0.3}$	$30.8_{\pm 0.6}$	$3.0_{\pm 0.0}$	$6.5_{\pm 0.4}$	$\mathbf{2.7_{\pm 0.4}}$	$10.3_{\pm 0.1}$	$9.1_{\pm 0.2}$	$13.0_{\pm 1.7}$	$6.7_{\pm 0.1}$	$9.9_{\pm 7.3}$
ESN (deep)	$\mathbf{9.7_{\pm 0.2}}$	$30.5_{\pm 0.3}$	$\mathbf{2.0_{\pm 0.0}}$	$\mathbf{4.2_{\pm 0.2}}$	$3.0_{\pm 0.5}$	$\mathbf{10.1_{\pm 0.1}}$	$8.9_{\pm 0.1}$	$\mathbf{9.6_{\pm 0.5}}$	$6.6_{\pm 0.0}$	$6.0_{\pm 0.6}$
ParalESN	$10.4_{\pm 0.5}$	$30.2_{\pm 0.5}$	$2.8_{\pm 0.2}$	$7.4_{\pm 0.4}$	$3.7_{\pm 0.8}$	$10.2_{\pm 0.1}$	$9.0_{\pm 0.2}$	$12.8_{\pm 1.2}$	$\mathbf{6.5_{\pm 0.1}}$	$5.1_{\pm 0.1}$
ParalESN (deep)	$10.3_{\pm 0.3}$	$\mathbf{29.4_{\pm 0.3}}$	$2.6_{\pm 0.4}$	$5.2_{\pm 0.5}$	$4.5_{\pm 1.0}$	$10.2_{\pm 0.2}$	$\mathbf{8.8_{\pm 0.1}}$	$9.7_{\pm 0.9}$	$\mathbf{6.5_{\pm 0.0}}$	$\mathbf{5.0_{\pm 0.0}}$

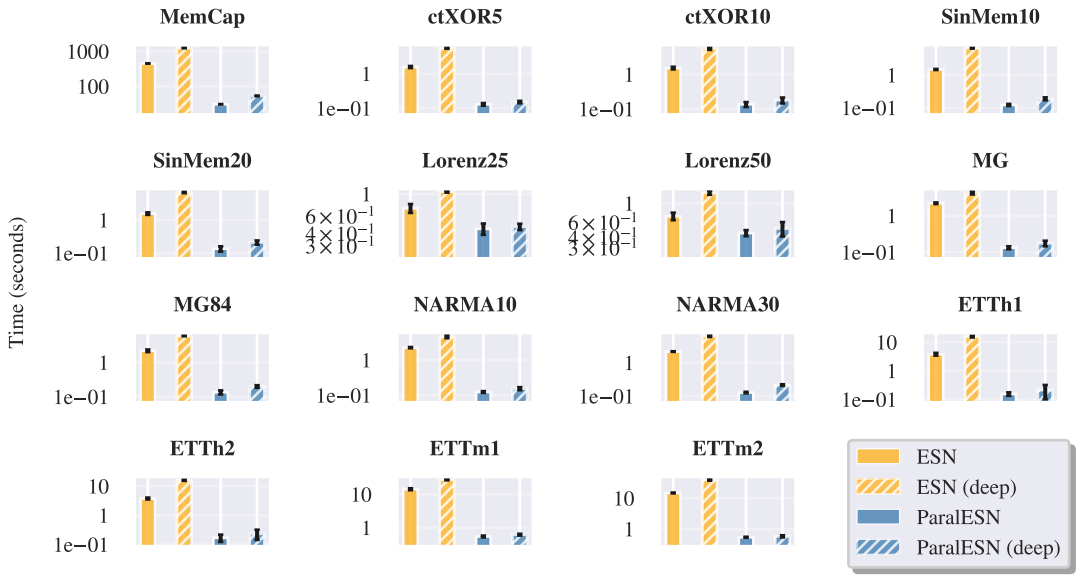

Figure 4. Training time comparison between ParalESNs and traditional ESNs for each memory-based and forecasting benchmark. ParalESN and ParalESN (deep) train order of magnitude faster.

+3.3% on StarLightCurves. Similarly, ParalESN (deep) outperforms ESN (deep) by +10.0% on Blink, +8.8% on FaultDetectionA, +2.7% on FordA, +0.7% on FordB, and +2.3% on StarLightCurves. On sMNIST and psMNIST, ParalESN achieves higher test accuracy by +13.4% and +17.1%, respectively, compared to traditional ESN. ParalESN (deep) provides gains of +7.7% and +13.1% over ESN (deep). These performance improvements come alongside substantial efficiency gains, with ParalESN and ParalESN (deep) requiring half or less of the training time, CO2 emissions, and energy of their counterparts. Unlike traditional RC, ParalESN is competitive with LSTMs, Transformers, LRU, and Mamba, while delivering computational savings of orders of magnitude.

6. Conclusions

We introduced ParalESN, a novel framework for constructing high-dimensional, efficient, and parallelizable untrained RNNs based on linear diagonal recurrence. This work addresses fundamental limitations of traditional RC, including the necessity of processing data sequentially and the prohibitive memory footprint of high-dimensional reservoirs. Results across various time series and 1-D pixel-level classification benchmarks demonstrate that ParalESN is, on average, more accurate and faster than traditional RC, while remaining competitive with fully-trainable sequence models at a fraction of their computational cost.

Table 3. Test set results on time series classification tasks, assuming 1024 recurrent neurons for each model. The **best result** is highlighted in bold.

MODEL	BLINK	FAULTDETECTIONA	FORDA	FORDB	STARLIGHTCURVES
ESN	$68.9_{\pm 5.5}$	$71.3_{\pm 0.8}$	$75.8_{\pm 0.9}$	$62.7_{\pm 0.8}$	$92.1_{\pm 0.7}$
ESN (DEEP)	$86.5_{\pm 1.8}$	$86.0_{\pm 0.6}$	$90.1_{\pm 0.8}$	$76.1_{\pm 0.8}$	$93.8_{\pm 0.4}$
PARALESN	$\mathbf{96.8_{\pm 1.1}}$	$75.0_{\pm 0.4}$	$83.4_{\pm 0.7}$	$68.4_{\pm 1.0}$	$95.4_{\pm 0.4}$
PARALESN (DEEP)	$96.5_{\pm 0.4}$	$\mathbf{94.8_{\pm 0.9}}$	$\mathbf{92.8_{\pm 0.5}}$	$\mathbf{76.8_{\pm 1.6}}$	$\mathbf{96.1_{\pm 0.3}}$

Table 4. Test set results sMNIST and psMNIST tasks. The top-two results are underlined, the **best result** overall is highlighted in bold.

SMNIST

MODEL	PARAMS.	↑ ACCURACY	↓ TIME (MIN.)	↓ EMISSIONS (KG)	↓ ENERGY (KWH)
LSTM	≈ 160k	$97.5_{\pm 1.4}$	$80.8_{\pm 6.8}$	$0.34_{\pm 0.14}$	$1.02_{\pm 0.42}$
TRANSFORMER	≈ 160k	$98.4_{\pm 0.1}$	$141.0_{\pm 14.1}$	$0.60_{\pm 0.28}$	$1.81_{\pm 0.86}$
LRU	≈ 160k	$\mathbf{98.5_{\pm 0.2}}$	$29.1_{\pm 1.85}$	$0.18_{\pm 0.02}$	$0.57_{\pm 0.05}$
MAMBA	≈ 200k	$98.4_{\pm 0.1}$	$22.87_{\pm 4.48}$	$0.19_{\pm 0.02}$	$0.57_{\pm 0.06}$
ESN	≈ 160k	$82.5_{\pm 7}$	$4.3_{\pm 0.1}$	$0.02_{\pm 0.00}$	$0.07_{\pm 0.00}$
ESN (DEEP)	≈ 160k	$91.4_{\pm 1.1}$	$8.8_{\pm 0.1}$	$0.04_{\pm 0.00}$	$0.13_{\pm 0.00}$
PARALESN	≈ 160k	$96.2_{\pm 1.3}$	$\mathbf{2.7_{\pm 0.7}}$	$\mathbf{0.01_{\pm 0.00}}$	$\mathbf{0.04_{\pm 0.1}}$
PARALESN (DEEP)	≈ 160k	$97.2_{\pm 0.2}$	$3.3_{\pm 0.5}$	$0.02_{\pm 0.00}$	$0.05_{\pm 0.00}$

PSMNIST

MODEL	PARAMS.	↑ ACCURACY	↓ TIME (MIN.)	↓ EMISSIONS (KG)	↓ ENERGY (KWH)
LSTM	≈ 160k	$92.8_{\pm 0.5}$	$89.3_{\pm 4.2}$	$0.47_{\pm 0.04}$	$1.41_{\pm 0.11}$
TRANSFORMER	≈ 160k	$97.4_{\pm 0.2}$	$156.8_{\pm 2.7}$	$0.65_{\pm 0.24}$	$1.98_{\pm 0.73}$
LRU	≈ 160k	$\mathbf{97.8_{\pm 0.1}}$	$33.8_{\pm 3.42}$	$0.21_{\pm 0.03}$	$0.63_{\pm 0.09}$
MAMBA	≈ 200k	$92.6_{\pm 0.1}$	$43.25_{\pm 5.02}$	$0.24_{\pm 0.03}$	$0.73_{\pm 0.08}$
ESN	≈ 160k	$78.2_{\pm 1.6}$	$4.3_{\pm 0.1}$	$0.02_{\pm 0.00}$	$0.06_{\pm 0.00}$
ESN (DEEP)	≈ 160k	$82.1_{\pm 3.7}$	$7.3_{\pm 0.1}$	$0.04_{\pm 0.00}$	$0.11_{\pm 0.00}$
PARALESN	≈ 160k	$96.9_{\pm 0.1}$	$\mathbf{2.8_{\pm 0.3}}$	$\mathbf{0.01_{\pm 0.00}}$	$\mathbf{0.04_{\pm 0.00}}$
PARALESN (DEEP)	≈ 160k	$95.2_{\pm 0.1}$	$3.1_{\pm 0.2}$	$0.02_{\pm 0.00}$	$0.05_{\pm 0.00}$

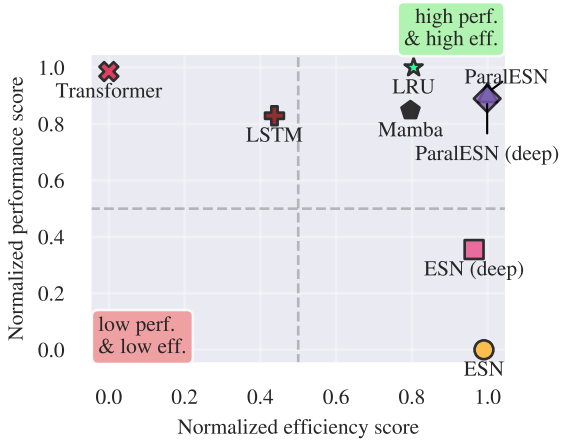

Figure 5. Trade-off between performance (test accuracy) and efficiency (training time) for ParalESN, traditional RC, and fully-trainable sequence models, for the sMNIST and psMNIST benchmarks. For each model and benchmark, we compute the percentage improvement over the ESN baseline. The normalized scores are then obtained via min-max normalization of these average improvements, mapping them to a $[0, 1]$ scale with 0 representing to the worst-performing model and 1 the best-performing one. ParalESN is competitive with fully-trainable models while delivering substantial efficiency improvements.